

Does Your Mobile App Work Without an Internet Connection?

Native mobile apps act as the interface between the user and the users' data and require uninterrupted connectivity. A native app with offline capability stores both the mobile app's software and its data locally on the mobile device. Offline mobile apps allow the user to run the app, regardless of connectivity.

How many times have you opened a mobile app, filled out a lengthy form that runs across multiple screens, and passed through validation errors only to see a message stating, "No Internet connection" when you finally click *Submit*? Most mobile applications are simply clients that send and receive data in real-time and become unusable without an active Internet connection.

As per the ICT Facts and Figures 2016, only 40.9 per cent of the population in developing countries has a mobile broadband subscription. And those who have it often complain about fluctuations, especially in crowded places or while travelling. Even in metro cities, the continuity of the network is not perfect. With unreliable networks, limited bandwidth and high latency, it becomes really challenging for engineering teams to come up with solutions that enable users to use an app, regardless of network connectivity. An approach where you make a network request, wait for the response and then display it, is quite unappealing. Ideally, you should have the data already, which you can display immediately, with a separate mechanism to update it.



the data. Choosing the right one is important and depends on the type of data your app presents. For example, time-sensitive data like stock updates should be as recent as possible, while images or lists of locations can be updated less frequently.

Network first: Try to retrieve data from the server first, and if that is not possible because there is no network, retrieve data from the local cache (if available). The next successful network hit should update the local copy. This strategy is useful when you always want to show the latest and most updated information.

Local first: Retrieve data from the cache and display it immediately. At the same time, if the network is available, fetch data from the server in the background and update the view as well as the cache. This approach is very useful as it leads to minimum in-app latency and the user gets to see the data instantly.

Local only: Cache data for a specific period and fetch it only from the cache without contacting the server at all. The cache can be later updated via notifications or periodic service polls. Do take into account the risk of battery drain caused by aggressive polling.

Prefetch: In some cases, it makes sense to download lightweight content and cache it pre-emptively so that it is instantly available when requested later.

Usually, a combination of these methods is required for different types of data at different places in the app. For example, product details can be cached for long, using either the 'local first' or the 'local only' strategy, whereas product price can be fetched in real-time by always using the 'network first' strategy. The longer the data can be cached, the better it is. Just be conscious of the size and security of data being stored. All sensitive data should be encrypted and stored safely.

API support

Although the clients can decide on, choose and implement their own caching policy, the complete benefit of caching can be realised only when complemented with some sort of server-side caching support as well. By sending headers like cache-control with appropriate values, the server can indicate

Adding the offline capability to apps

Here are a couple of questions that must be answered before taking the plunge. What are the key features of the app that render it unusable due to no/flaky network conditions? How many users are affected due to this? How does it affect the key metrics of your business? Answers to these will help reveal the areas where there is an opportunity to provide offline support. There may not be a need to take the complete app offline. Features that cannot work offline can be redesigned, disabled or even be hidden.

Offline views

In order for the app to function in unsteady and offline environments, the data that the application needs must be persisted locally on the device for subsequent lookups.

Caching

There could be different strategies for caching and refreshing